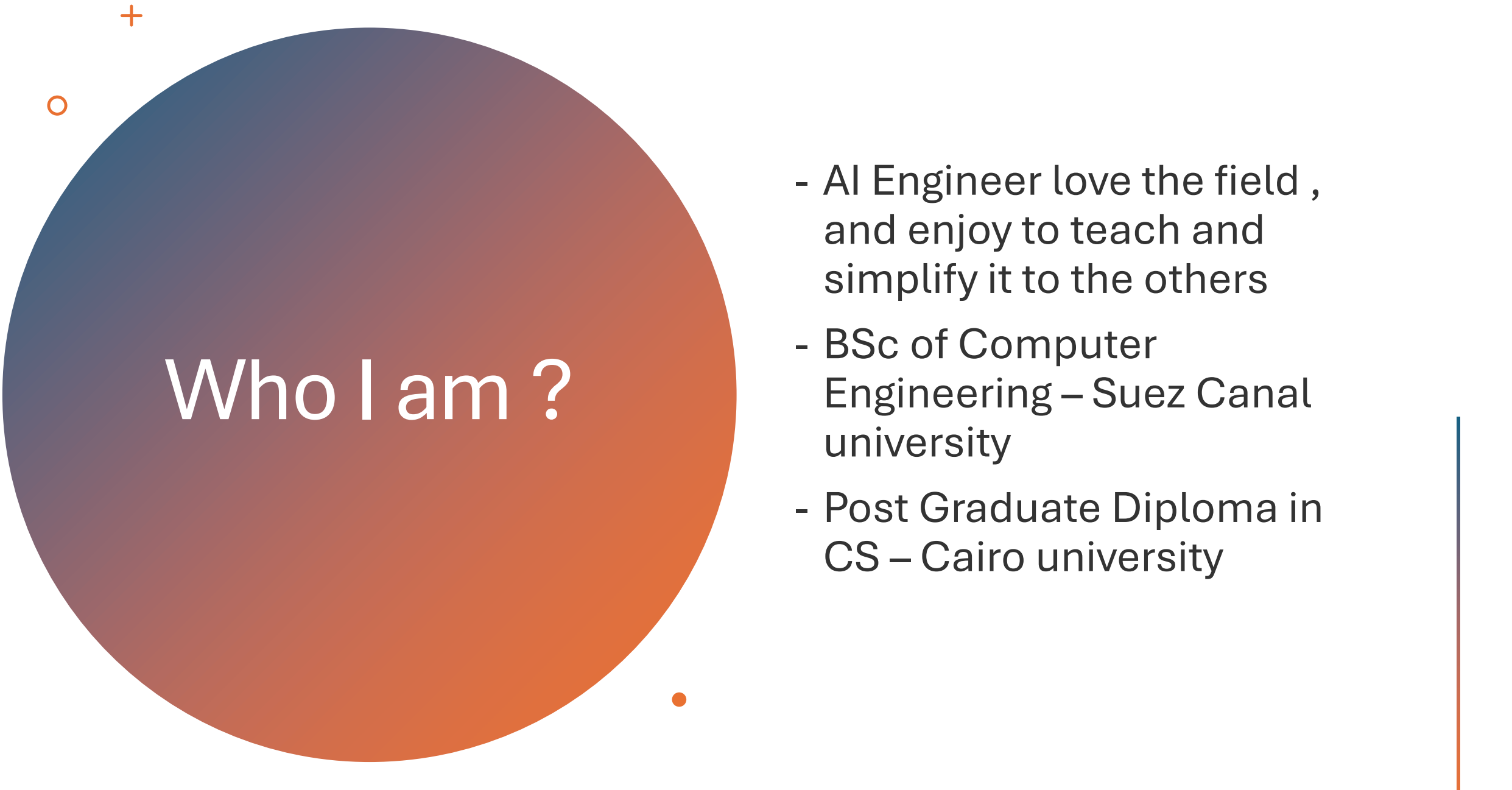


Transformers

Eng : Mohammed Sayed Hemed

Eng :
Mohammed
S. Hemed





Who I am ?

- AI Engineer love the field ,
and enjoy to teach and
simplify it to the others
- BSc of Computer
Engineering – Suez Canal
university
- Post Graduate Diploma in
CS – Cairo university



Course Objectives

- Digest Transformers , HF , HF Hub.
 - How to search for model and use it ?
 - How to make your own model if it isn't exist ?
 - How to search for dataset ?
 - How to make your own dataset if it isn't exist ?
 - How to run any model (audio – image - text) with just two lines of Code?
 - What happened behind the scenes ?
 - What are Transformer different Architectures?
 - How to Finetune pretrained model ?
 - How to Finetune LLMs ?
 - How to understand model learning curves ?
 - Mainly, this course is tailored to your interests,
- Depending on you**, we will determine the depth and breadth of what we dive.
- 



Prerequisites

- What is the building block of DL Model ? Tensors
 - Familiarity with Python language , Pytorch DL framework.
- 

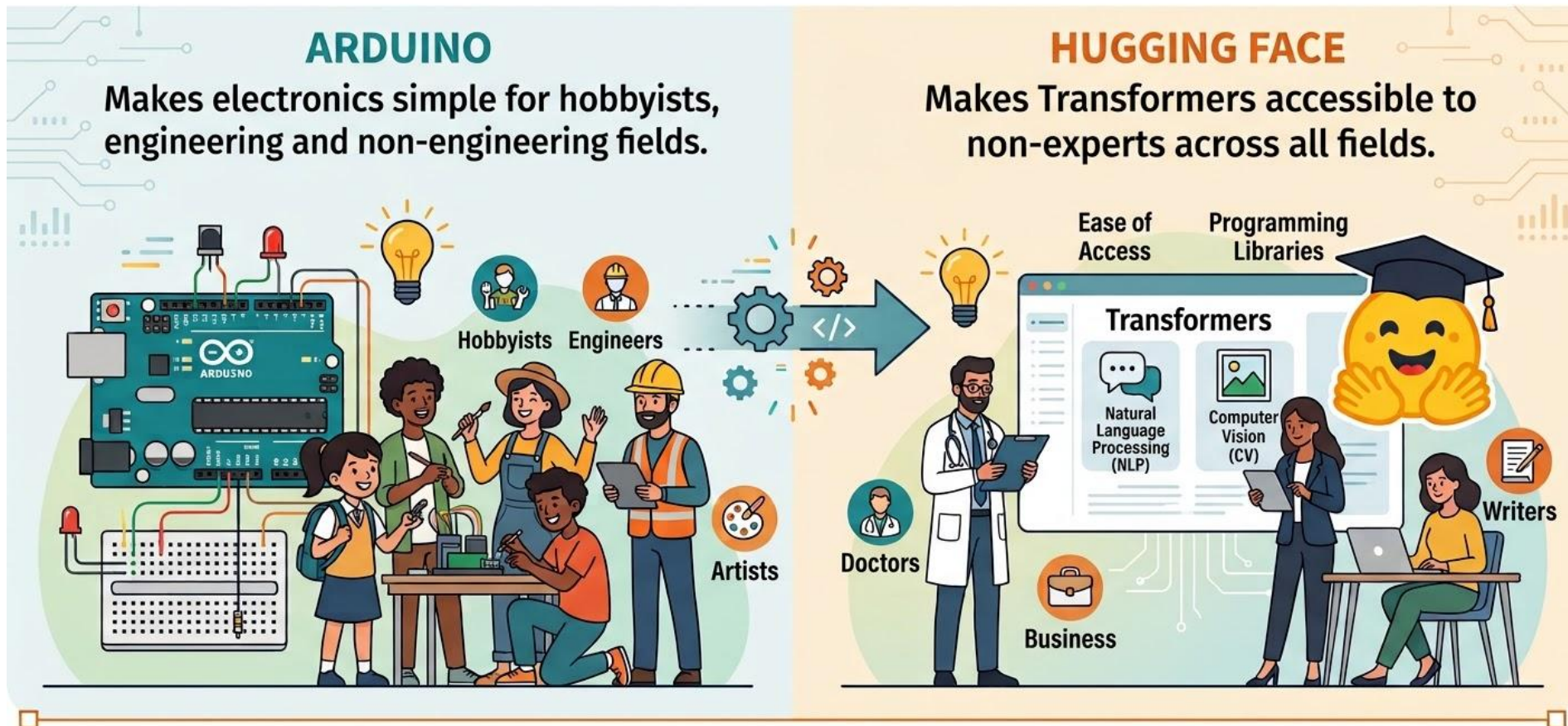
What is Hugging Face look like ?

- It's look like the Arduino platform which make embedded system and electronics available to anyone with no pre-requisites.
- It give a strong tool at any field to non specialist in the AI to get the most benefit from these models to help them in other fields like medicine , industry , ... etc.
- Just make an account on HF you enjoy millions of free models you can use and deploy it , huge number of datasets , and a space through it you can host and display your own model.

Hugging Face turn Transformers into LEGO !!



This is The AI Arduino platform !





What is Hugging Face ?

- **The GitHub for Machine Learning & AI Collaboration**
 - **Centralized AI Repository**
 - Hosts open-access **Models, Datasets**, and **ML Applications** (Spaces) in one place.
 - **Core Pillar Assets**
 - **Model Hub:** over 3M pre-trained models (LLMs, Computer Vision, Audio, Multimodal).
 - **Dataset Hub:** Thousands of curated, community-driven datasets for training and benchmarking.
 - **Spaces:** Interactive web applications and demos built with Streamlit or Gradio to showcase models.
 - **Version Control & Collaboration**
 - Powered by **Git & Git LFS** to manage large model weights, tracks changes, and enable branch-based collaboration.
 - **Seamless Ecosystem Integration**
 - Natively integrates with major ML libraries (transformers, diffusers, PyTorch, TensorFlow, vLLM).
- 

What is Hugging Face HUB

The screenshot displays the Hugging Face HUB interface. At the top, the navigation bar includes the Hugging Face logo, a search bar, and links for Models, Datasets, Spaces, Docs, Pricing, and a user profile. The main content area is divided into two sections. The left sidebar contains navigation tabs (Main, Tasks, Libraries, Languages, Licenses, Other) and filters for Tasks (Text Generation, Any-to-Any, Image-Text-to-Text, Image-to-Text, Image-to-Image, Text-to-Image, Text-to-Video, Text-to-Speech) and Parameters (a slider from <1B to >500B). The right section shows a list of models with filters for Inference and sorting by Trending. The models listed are:

- convaiinnovations/laya**: Text Classification, 0.4B, Updated about 10 hours ago, 3.32k likes.
- Qwen/Qwen-Image-2.1**: Text-to-Image, 7B, Updated 3 days ago, 37.6k downloads, 2.15k likes.
- prism-ml/Ternary-Bonsai-2-27B-gguf**: Text Generation, 27B, Updated about 11 hours ago, 2.99M downloads, 2k likes.
- XingChen-AGI/Xing4.0-29B-A4B**: Text Generation, 31B, Updated 6 days ago, 41.9k downloads, 1.64k likes.
- abenzperps/Qwen-Image-2.1-Uncensored-GGUF**: Text-to-Image, 7B, Updated about 24 hours ago, 576k downloads, 1.58k likes.

Chess AI Model !!

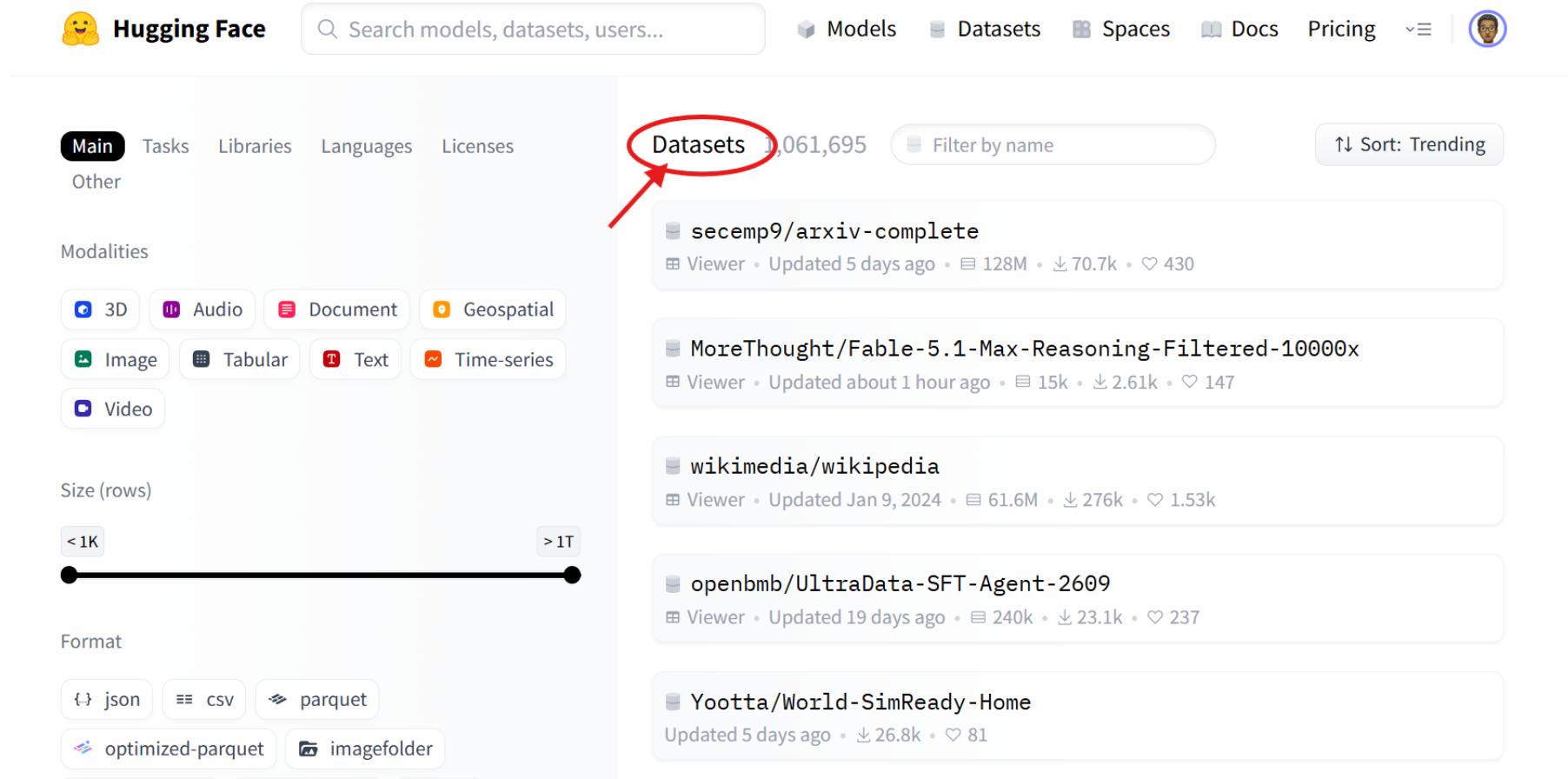


Chess AI Model !!

- This Model plays like 2000+ rating player which is difficult to beat.
- The model size is 230M !!
- You can play vs this model in this Hugging Face space :
<https://huggingface.co/spaces/mlabonne/ChessLFM>
- It uses transformer.js library from HF , this library allows you to run models locally in web browser using your GPU , you can the speed and performance in the gif.



What is Hugging Face HUB



The screenshot displays the Hugging Face HUB interface. At the top, the Hugging Face logo is on the left, followed by a search bar labeled "Search models, datasets, users...". Navigation links for "Models", "Datasets", "Spaces", "Docs", and "Pricing" are in the center, and a user profile icon is on the right. The left sidebar contains filters for "Main" (selected), "Tasks", "Libraries", "Languages", and "Licenses". Below these are "Modalities" (3D, Audio, Document, Geospatial, Image, Tabular, Text, Time-series, Video) and a "Size (rows)" slider ranging from "< 1K" to "> 1T". The "Format" section includes buttons for "json", "csv", "parquet", "optimized-parquet", and "imagefolder". The main content area is titled "Datasets" (circled in red with an arrow) and shows "1,061,695" datasets. It includes a "Filter by name" input and a "Sort: Trending" button. A list of datasets is displayed, each with a viewer icon, name, update time, size, and likes.

Hugging Face Search models, datasets, users... Models Datasets Spaces Docs Pricing

Main Tasks Libraries Languages Licenses Other

Modalities

3D Audio Document Geospatial Image Tabular Text Time-series Video

Size (rows)

< 1K > 1T

Format

json csv parquet optimized-parquet imagefolder

Datasets 1,061,695 Filter by name Sort: Trending

- secemp9/arxiv-complete
Viewer • Updated 5 days ago • 128M • 70.7k • 430
- MoreThought/Fable-5.1-Max-Reasoning-Filtered-10000x
Viewer • Updated about 1 hour ago • 15k • 2.61k • 147
- wikimedia/wikipedia
Viewer • Updated Jan 9, 2024 • 61.6M • 276k • 1.53k
- openbmb/UltraData-SFT-Agent-2609
Viewer • Updated 19 days ago • 240k • 23.1k • 237
- Yootta/World-SimReady-Home
Updated 5 days ago • 26.8k • 81

Setup

- `!pip install transformers[sentencepiece]`
- Cache model to google drive instead of every time download model weights and lost time

Transformers, what can they do?

The most basic object in the 🤗 Transformers library is the pipeline() function. It connects a model with its necessary preprocessing and postprocessing steps, allowing us to directly input any text and get an intelligible answer:

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")
classifier("I've been waiting for a HuggingFace course my whole life.")

[{'label': 'POSITIVE', 'score': 0.9598047137260437}]
```

We can even pass several sentences!

```
classifier(
    ["I've been waiting for a HuggingFace course my whole life.", "I hate this so much!"]
)
```

```
[{'label': 'POSITIVE', 'score': 0.9598047137260437},
 {'label': 'NEGATIVE', 'score': 0.9994558095932007}]
```

Ask HuggingChat

By default, this pipeline selects a particular pretrained model that has been fine-tuned for sentiment analysis in English. The model is downloaded and cached when you create the classifier object. If you rerun the command, the cached model will be used instead and there is no need to download the model again



Steps when pass a text to a pipeline



- The text is preprocessed into a format the model can understand.
 - The preprocessed inputs are passed to the model.
 - The predictions of the model are post-processed, so you can make sense of them.
- 



Available pipelines for different modalities



Text pipelines :

text-generation - text-classification –
summarization – translation

zero-shot-classification : Classify text
without prior training on specific labels-

feature-extraction : Extract vector
representations of text

- **Image pipelines :**

image-to-text - image-classification -
object-detection

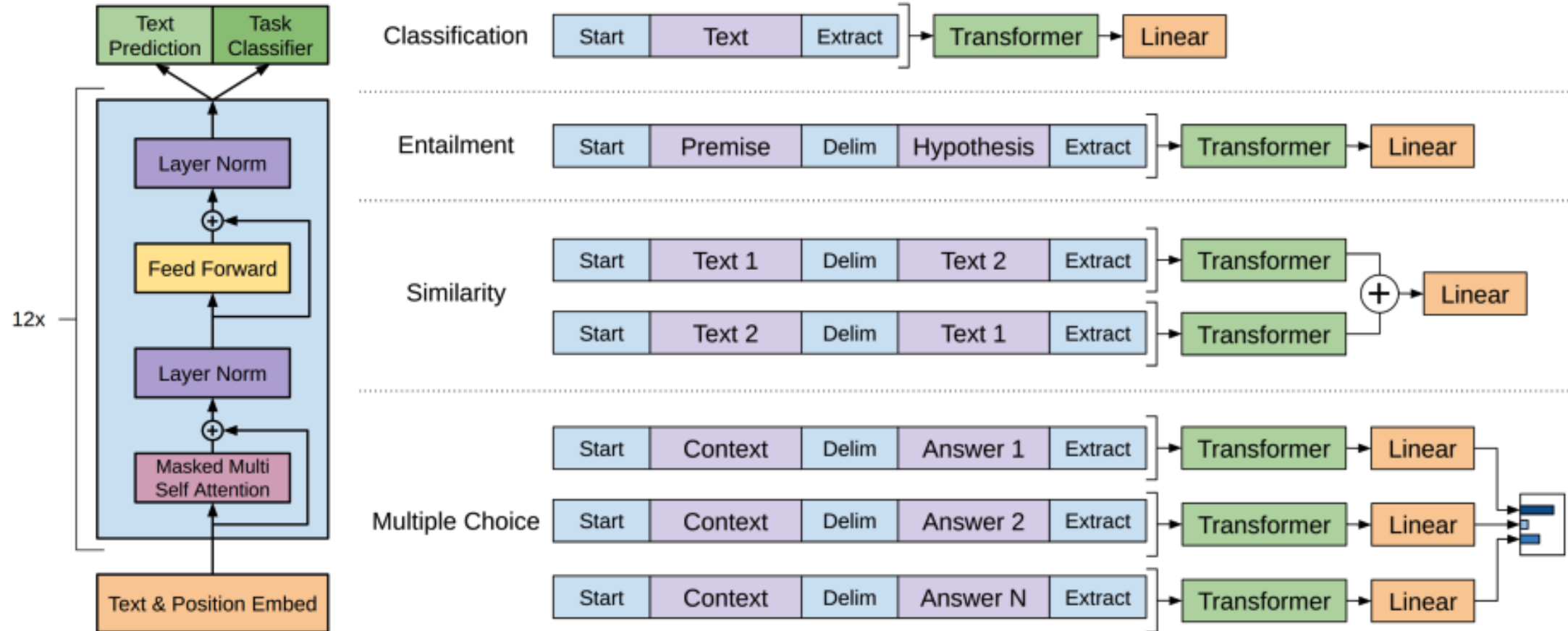
- **Audio pipelines :**

automatic-speech-recognition - audio-
classification - text-to-speech TTS

- **Multimodal pipelines**

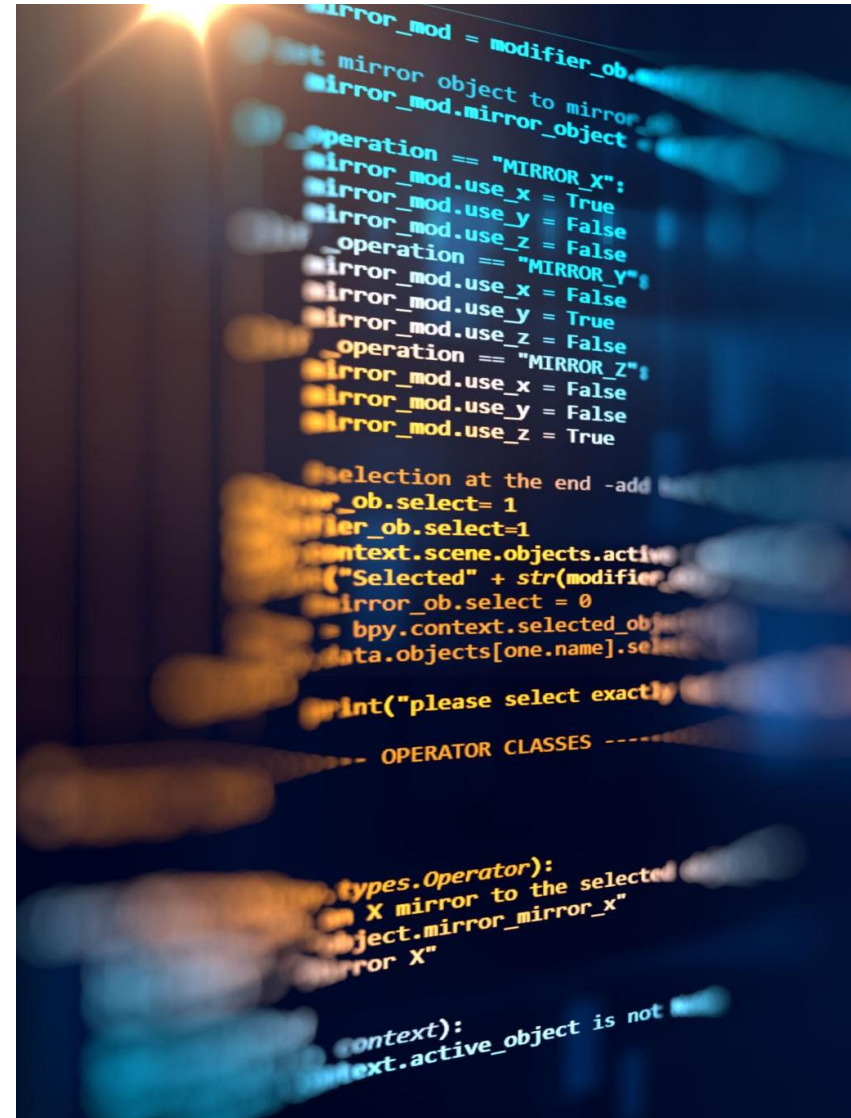
- image-text-to-text: Respond to an image
based on a text prompt
- 

How transformers Solve Tasks ?



Zero-shot classification

- a more challenging task where we need to classify texts that haven't been labelled.
- This is **a common scenario in real-world projects because annotating text is usually time-consuming** and requires domain expertise.
- For this use case, the zero-shot-classification pipeline is very powerful: it allows you to specify which labels to use for the classification,
- so you don't have to rely on the labels of the pretrained model. You've already seen how the model can classify a sentence as positive or negative using those two labels — but it can also classify the text using any other set of labels you like.
- This pipeline is called **zero-shot** because you don't need **to fine-tune the model on your data** to use it. It can directly return probability scores for any list of labels you want!



```
from transformers import pipeline
```

```
classifier = pipeline("zero-shot-classification")
```

```
classifier(  
    "This is a course about the Transformers library",  
    candidate_labels=["education", "politics", "business"],  
)
```

```
{  
    'sequence': 'This is a course about the Transformers library',  
    'labels': ['education', 'business', 'politics'],  
    'scores': [0.8445963859558105, 0.111976258456707, 0.043427448719739914]}  
}
```

Text generation

- Text generation involves creating coherent and contextually relevant text based on a prompt or input.
- [GPT-2](#) is a decoder-only model pretrained on a large amount of text. It can generate convincing (though not always true!) text given a prompt and complete other NLP tasks like question answering despite not being explicitly trained to.

Transformer models for language



LANGUAGE MODELS ARE AT THE HEART OF MODERN NLP. THEY'RE DESIGNED TO UNDERSTAND AND GENERATE HUMAN LANGUAGE BY LEARNING THE STATISTICAL PATTERNS AND RELATIONSHIPS BETWEEN WORDS OR TOKENS IN TEXT.



THE TRANSFORMER WAS INITIALLY DESIGNED FOR MACHINE TRANSLATION, AND SINCE THEN, IT HAS BECOME THE DEFAULT ARCHITECTURE FOR SOLVING ALL AI TASKS. SOME TASKS LEND THEMSELVES TO THE TRANSFORMER'S ENCODER STRUCTURE, WHILE OTHERS ARE BETTER SUITED FOR THE DECODER. STILL, OTHER TASKS MAKE USE OF BOTH THE TRANSFORMER'S ENCODER-DECODER STRUCTURE.



Transformers are language models

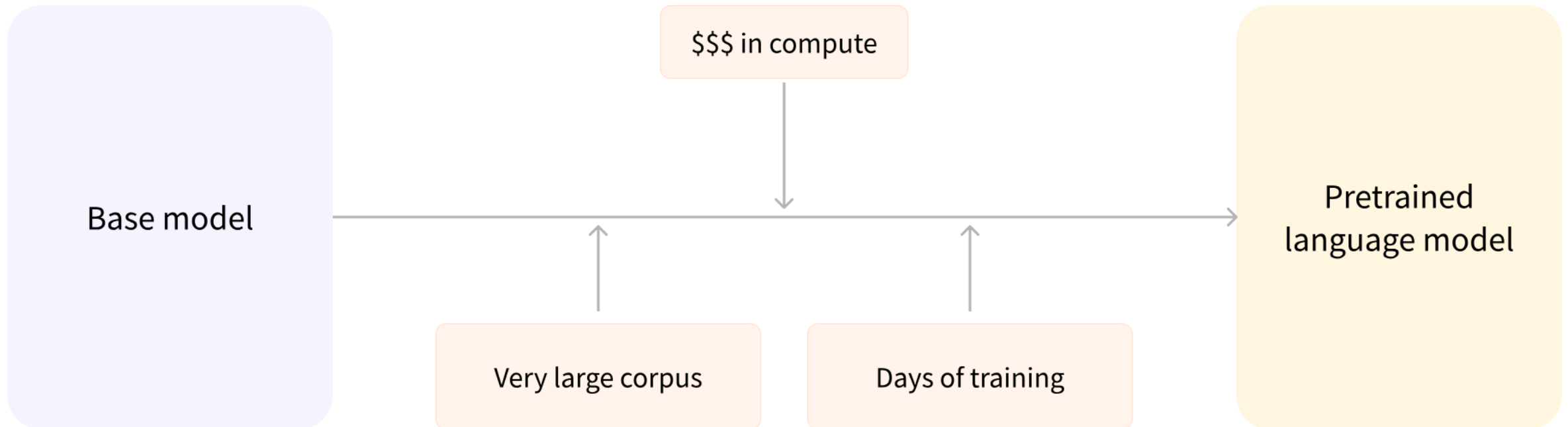


- All the Transformer models mentioned above (GPT, BERT, T5, etc.) have been trained as *language models*. This means they have been trained on large amounts of raw text in a self-supervised fashion.
 - Self-supervised learning is a type of training in which the objective is automatically computed from the inputs of the model. That means that humans are not needed to label the data!
 - This type of model develops a statistical understanding of the language it has been trained on, but it's less useful for specific practical tasks. Because of this, the general pretrained model then goes through a process called *transfer learning* or *fine-tuning*. During this process, the model is fine-tuned in a supervised way — that is, using human-annotated labels — on a given task.
- 

Ways of training Transformers

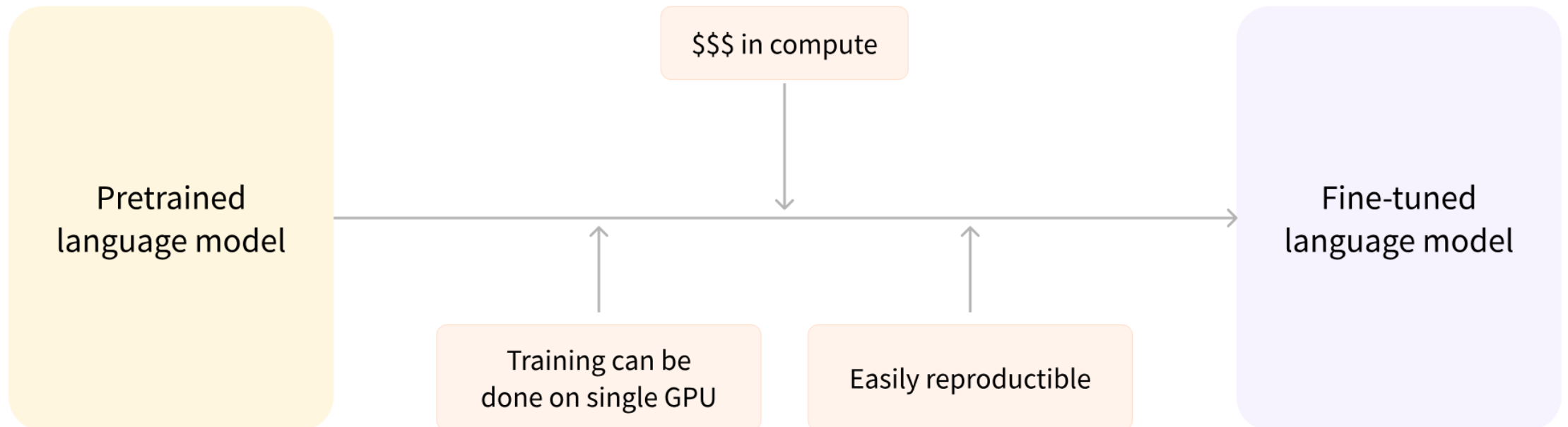
Pretraining

is the act of training a model from scratch: the weights are randomly initialized, and the training starts without any prior knowledge.



Ways of training Transformers

- *Fine-tuning*, on the other hand, is the training done **after** a model has been pretrained. To perform fine-tuning, you first acquire a pretrained language model, then perform additional training with a dataset specific to your task.





Wait — why not
simply train the
model for your
final use case
from the start
(**scratch**)?



- The pretrained model was already trained on a dataset that has some similarities with the fine-tuning dataset.
 - The fine-tuning process is thus able to take advantage of knowledge acquired by the initial model during pretraining (for instance, with NLP problems, the pretrained model will have some kind of statistical understanding of the language you are using for your task). Since the pretrained model was already trained on lots of data, the fine-tuning requires way less data to get decent results.
 - For the same reason, the amount of time and resources needed to get good results are much lower
- 

How language models work

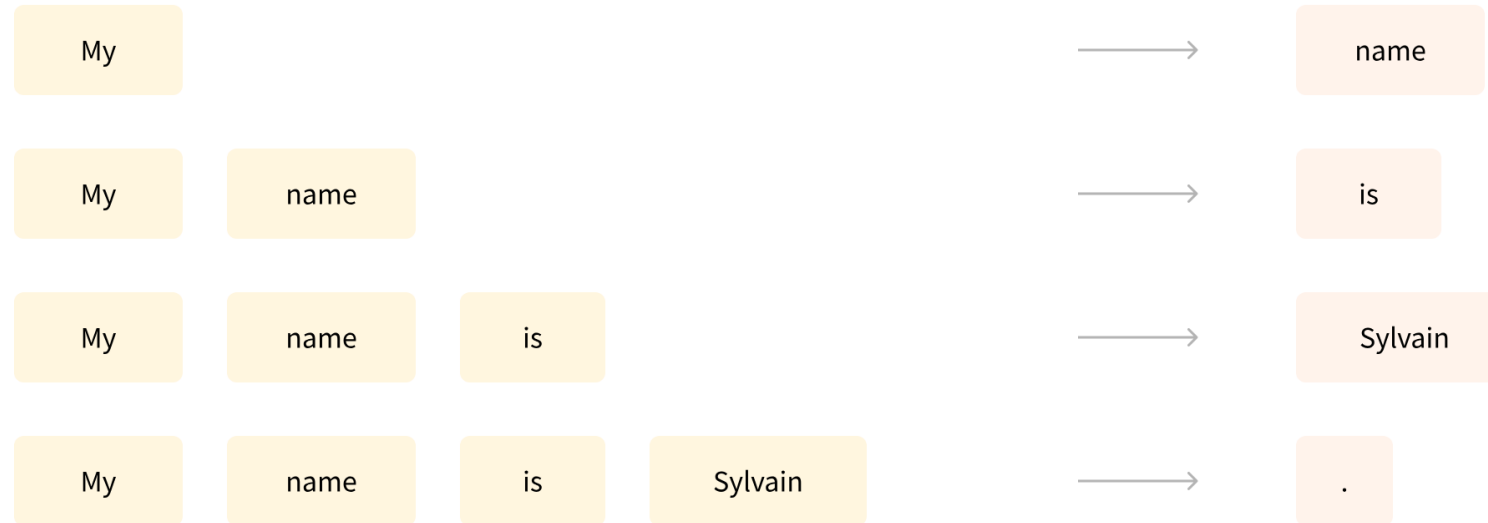
- Language models work by being trained to predict the probability of a word given the context of surrounding words. This gives them a foundational understanding of language that can generalize to other tasks.

How language models work

- There are two main approaches for [training a transformer model](#):

(1) Causal language modeling (MLM):

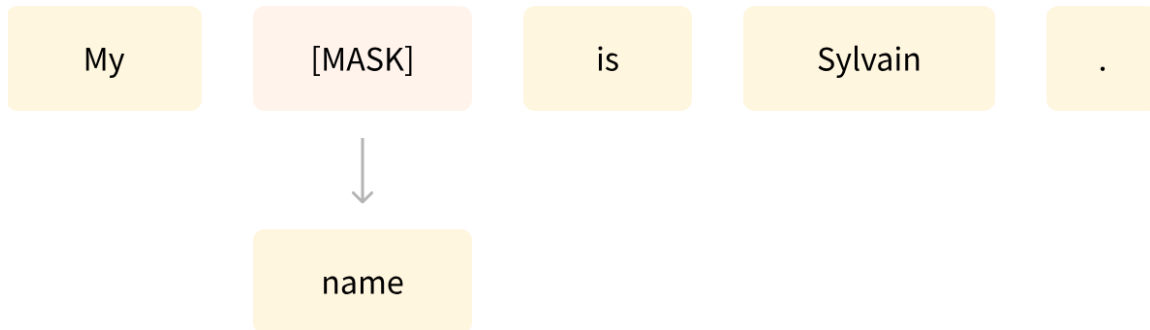
Used by decoder models like GPT, this approach predicts the next token based on all previous tokens in the sequence. The model can only use context from the left (previous tokens) to predict the next token



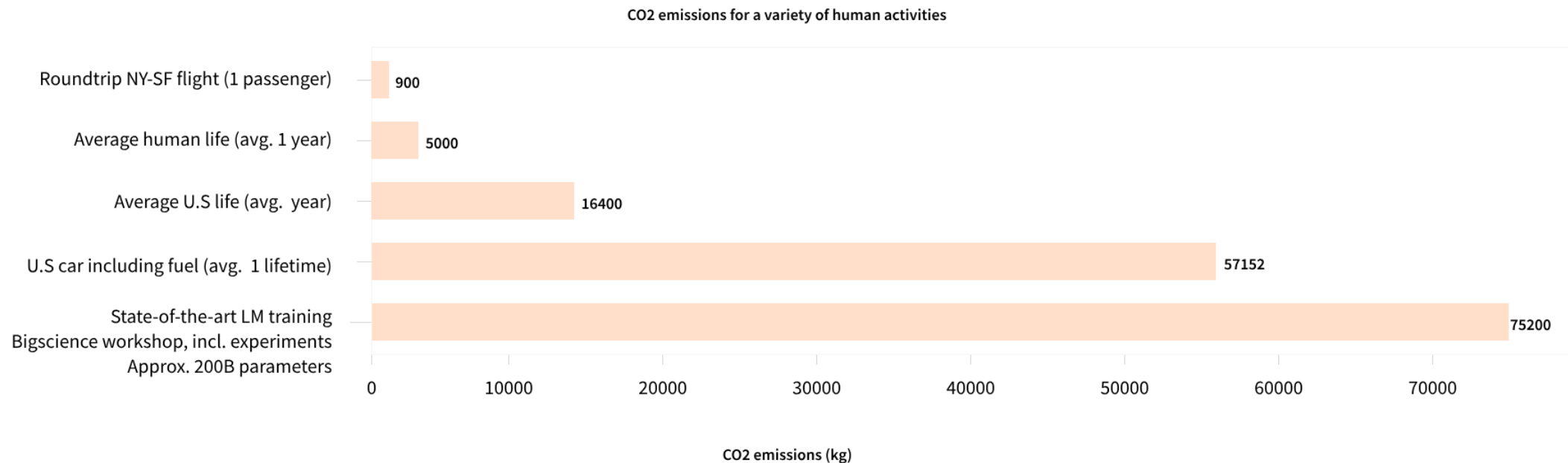
How language models work

(2) Causal language modeling (CLM):

Used by encoder models like BERT, this approach randomly masks some tokens in the input and trains the model to predict the original tokens based on the surrounding context. This allows the model to learn bidirectional context (looking at words both before and after the masked word).



Environmental Effect



+

•

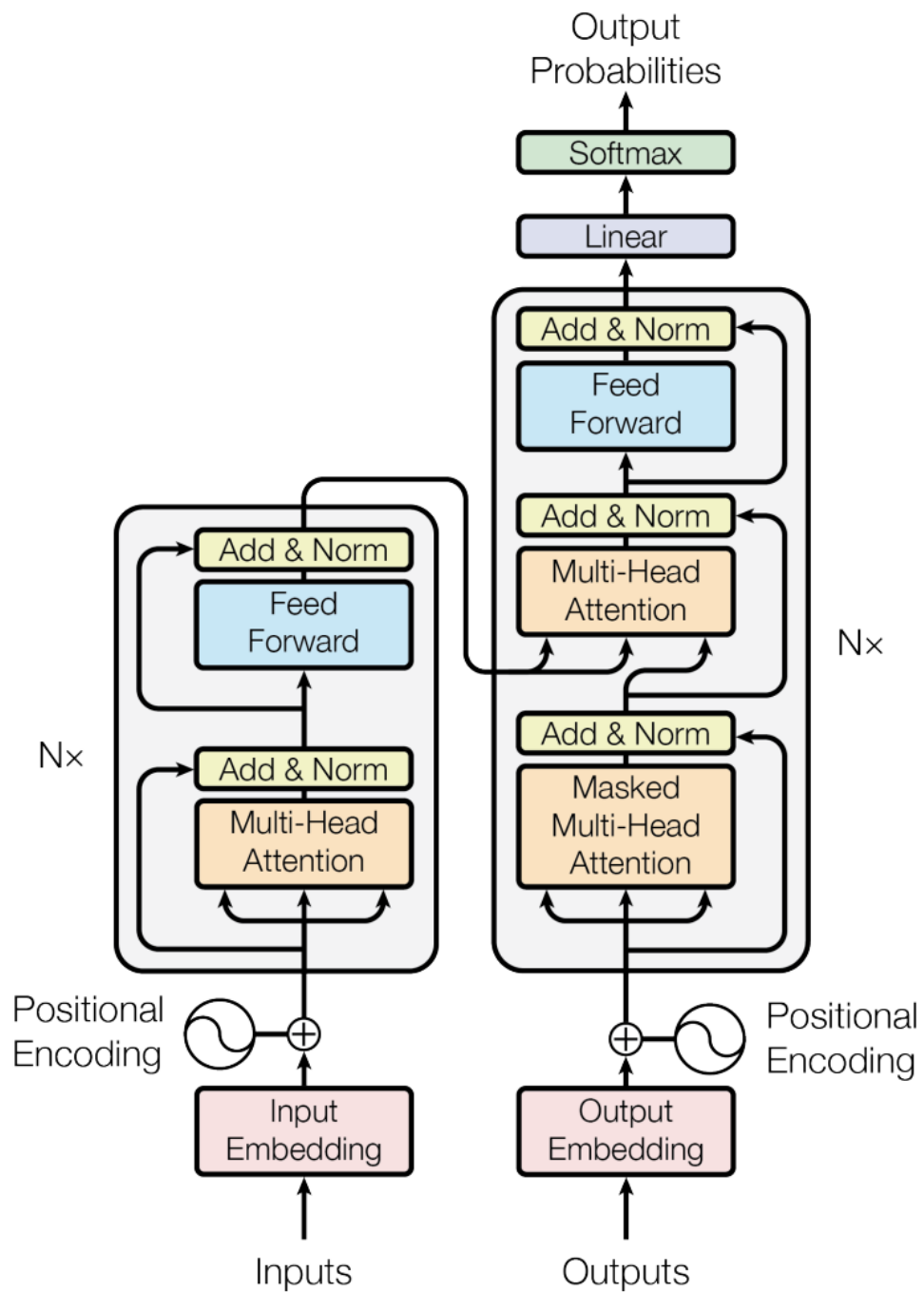
○

Types of language models

- In the Transformers library, language models generally fall into three architectural categories:
- **Encoder-only models** (like BERT): These models use a bidirectional approach to understand context from both directions. They're best suited for tasks that require deep understanding of text, such as classification, named entity recognition, and question answering.
- **Decoder-only models** (like GPT, Llama): These models process text from left to right and are particularly good at text generation tasks. They can complete sentences, write essays, or even generate code based on a prompt.
- **Encoder-decoder models** (like T5, BART): These models combine both approaches, using an encoder to understand the input and a decoder to generate output. They excel at sequence-to-sequence tasks like translation, summarization, and question answering.

BERT

Encoder



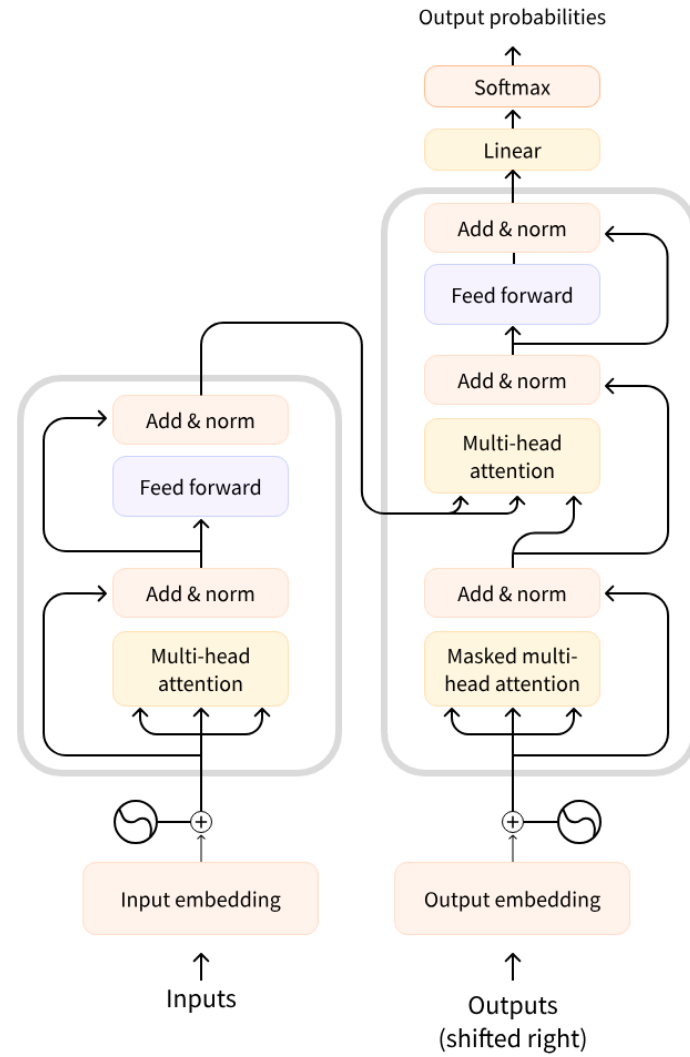
GPT

Decoder

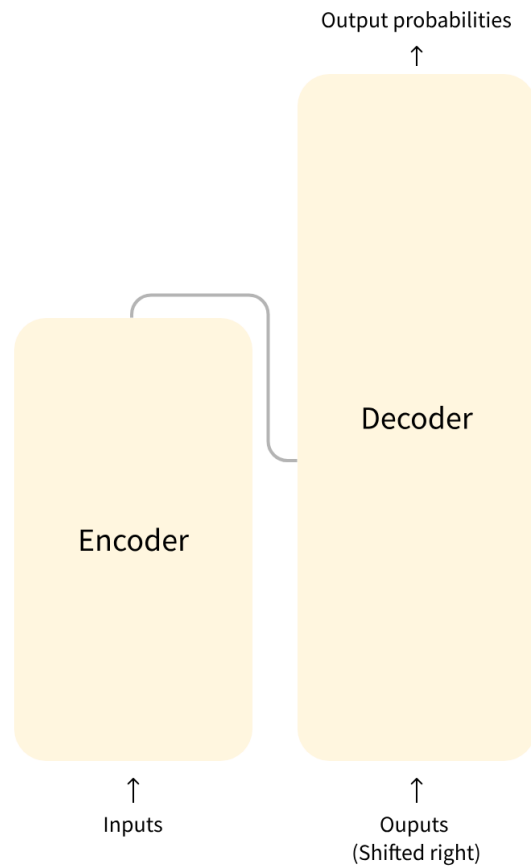
Choosing the right model

- Understanding which part of the Transformer architecture (encoder, decoder, or both) is best suited for a particular NLP task is key to choosing the right model.
- Generally :
- tasks requiring bidirectional context use **encoders**,
- tasks generating text use **decoders**,
- tasks converting one sequence to another use **encoder-decoders**.

Transformer Architecture



Transformer Archeticture





Modern Large Language Models (LLMs)



- Most modern Large Language Models (LLMs) use the **decoder-only** architecture. These models have grown dramatically in size and capabilities over the past few years, with some of the largest models containing hundreds of billions of parameters.
 - Modern LLMs are typically trained in two phases:
 - **Pretraining:** The model learns to predict the next token on vast amounts of text data
 - **Instruction tuning:** The model is fine-tuned to follow instructions and generate helpful responses
 - This approach has led to models that can understand and generate human-like text across a wide range of topics and tasks.
- 

Modern **decoder-based LLMs** have demonstrated impressive capabilities:

Capability	Description	Example
Text generation	Creating coherent and contextually relevant text	Writing essays, stories, or emails
Summarization	Condensing long documents into shorter versions	Creating executive summaries of reports
Translation	Converting text between languages	Translating English to Spanish
Question answering	Providing answers to factual questions	“What is the capital of France?”
Code generation	Writing or completing code snippets	Creating a function based on a description
Reasoning	Working through problems step by step	Solving math problems or logical puzzles
Few-shot learning	Learning from a few examples in the prompt	Classifying text after seeing just 2-3 examples

Encoder-Decoder (Seq2Seq) models excel at tasks that require transforming one form of text into another while preserving meaning. Some practical applications include:

Application	Description	Example Model
Machine translation	Converting text between languages	Marian, T5
Text summarization	Creating concise summaries of longer texts	BART, T5
Data-to-text generation	Converting structured data into natural language	T5
Grammar correction	Fixing grammatical errors in text	T5
Question answering	Generating answers based on context	BART, T5

Choosing the right architecture

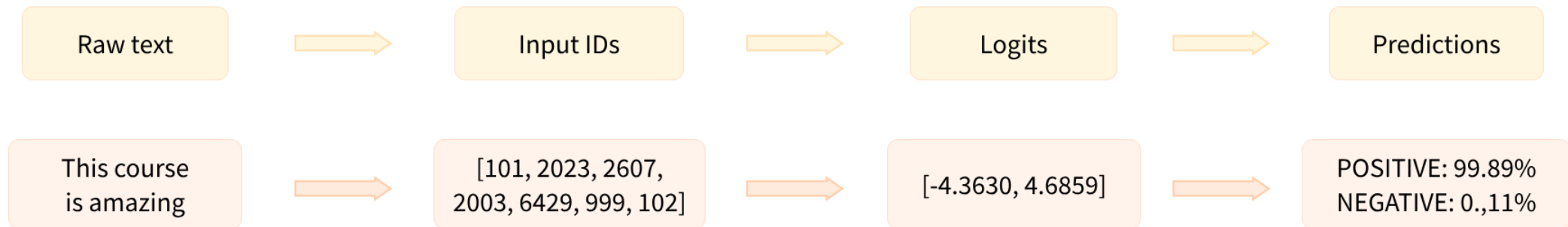
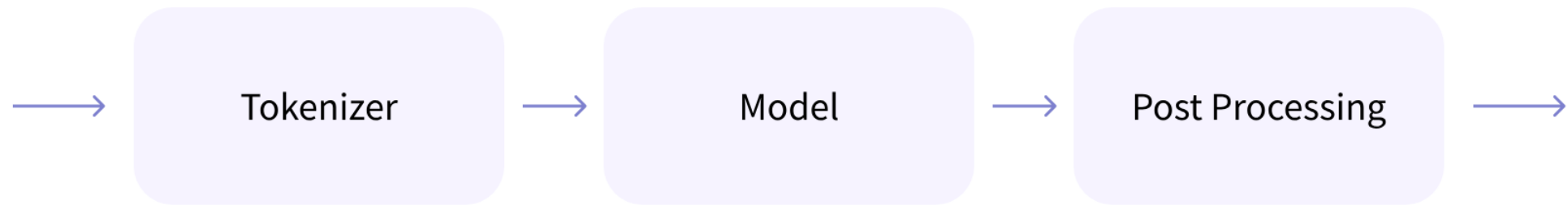
When working on a specific NLP task, how do you decide which architecture to use? Here's a quick guide:

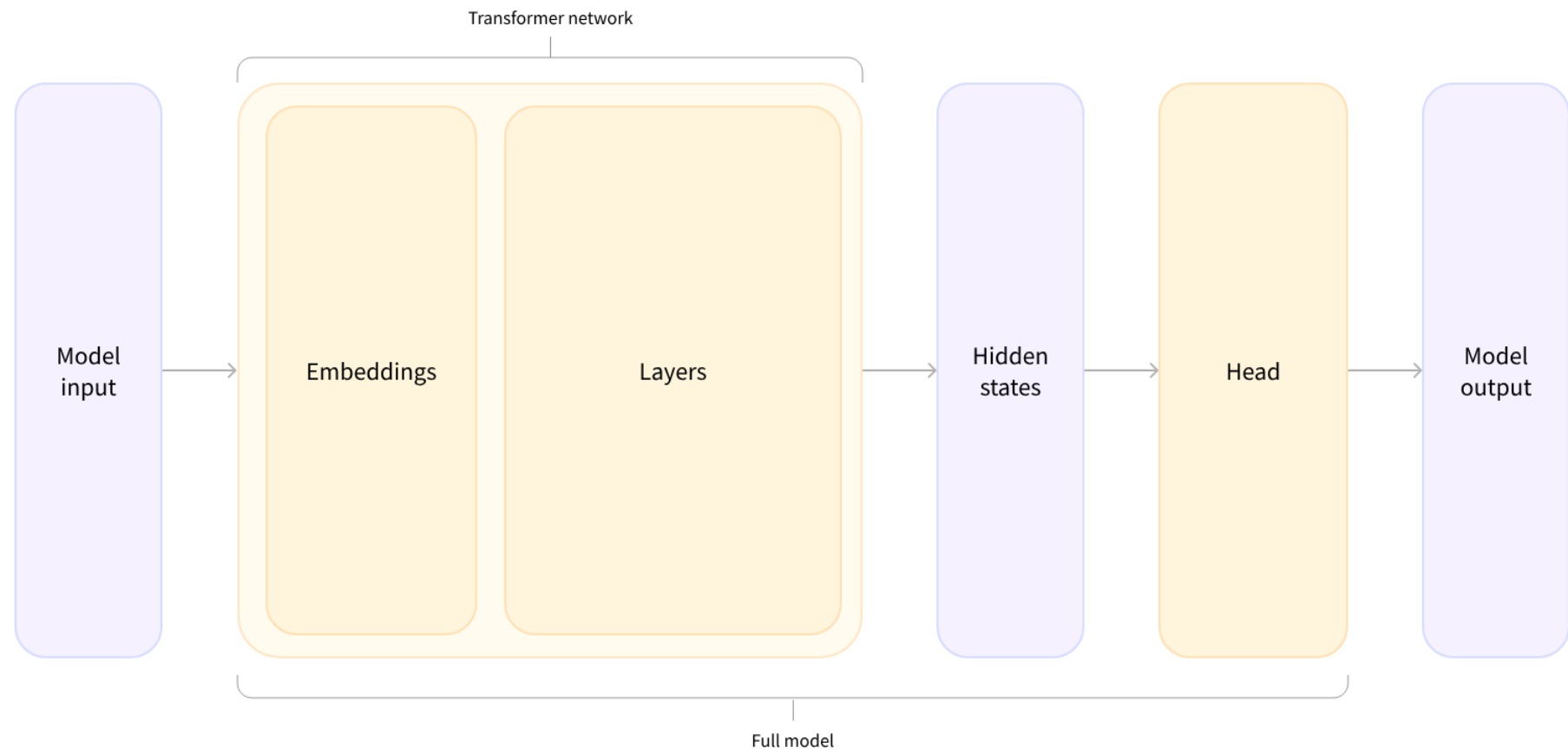
Task	Suggested Architecture	Examples
Text classification (sentiment, topic)	Encoder	BERT, RoBERTa
Text generation (creative writing)	Decoder	GPT, LLaMA
Translation	Encoder-Decoder	T5, BART
Summarization	Encoder-Decoder	BART, T5
Named entity recognition	Encoder	BERT, RoBERTa
Question answering (extractive)	Encoder	BERT, RoBERTa
Question answering (generative)	Encoder-Decoder or Decoder	T5, GPT
Conversational AI	Decoder	GPT, LLaMA

- +
-
- When in doubt about which model to use, consider:

- What kind of understanding does your task need? (Bidirectional or unidirectional)
- Are you generating new text or analyzing existing text?
- Do you need to transform one sequence into another?
- The answers to these questions will guide you toward the right architecture.

Behind pipeline()





Model

Model parameters(Weights) vs Model Hyperparameters

COMPARISON: MODEL PARAMETERS (WEIGHTS) vs. MODEL HYPERPARAMETERS

MODEL PARAMETERS (WEIGHTS & BIASES)

Definition

Learned internal attributes (the "brain cells")

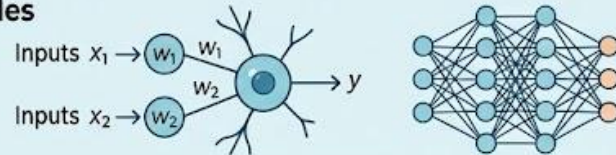
Origin

Discovered automatically by the training algorithm (e.g., via Gradient Descent)

Role

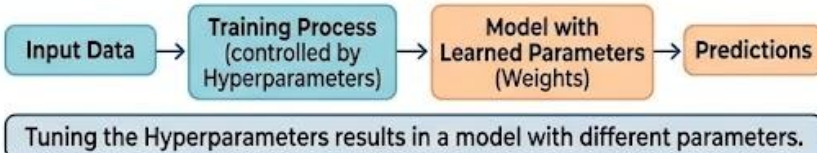
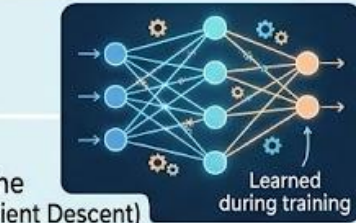
Define how the model transforms inputs to outputs

Examples



Key Insight

These *are* the model's "learned knowledge"



MODEL HYPERPARAMETERS

Definition

Configurable external settings (the "knobs & dials")

Origin

Set manually *before* training starts by the data scientist/engineer

Role

Control the overall behavior of the training process and architecture

Examples

- Number of Hidden Layers
- Number of Neurons per Layer
- Learning Rate (e.g., 0.001)
- Batch Size (e.g., 32)
- Number of Epochs (e.g., 10)



Key Insight

These *tune* how the model *learns* the parameters



Keywords

Keyword	Simple Definition
Epoch	One complete pass where the AI reads through the entire training dataset from start to finish.
Training	The process of teaching a model by letting it process data, make mistakes, and adjust its internal math to get better over time.
Inference	The phase when a fully trained model is put to work making actual predictions on brand new, unseen data.
Checkpoint	A saved snapshot of a model's weights and configuration at a specific point during or after training.
Weights / Parameters	The internal numbers (the model's "brain wiring") that are adjusted during training to determine how the model turns inputs into outputs.
Hyperparameters	Settings configured before training starts (e.g., learning rate, batch size) that control how the model learns.
Tokenizer	A tool that converts raw text into numbers (and vice versa) so the mathematical model can process it.
Tokens	The smaller pieces (words, subwords, or punctuation) that a tokenizer breaks text into before processing.
Logits	The raw, unnormalized numerical scores output by a model's final layer before they are converted into percentage probabilities.
Padding	Adding dummy filler tokens (like zeros) to shorter text inputs so all sequences in a batch are the exact same length.
Truncation	Cutting off long text sequences so they fit within the model's maximum allowed context window.
Attention Mask	A list of 1s and 0s that tells the model which tokens are real text (1) and which are filler padding (0) to ignore.
Encoder	The part of a Transformer that reads the full input text in both directions to build a rich contextual understanding.
Decoder	The part of a Transformer that generates output text word-by-word sequentially using causal masking to prevent looking ahead.
Distillation	Training a smaller, faster "student" model to mimic the outputs and performance of a larger "teacher" model.
Zero-Shot Learning	Getting a model to perform a task correctly without giving it any specific training examples for that exact task beforehand.
Few-Shot Learning	Giving a model a few examples inside the prompt to guide its understanding before asking for a new prediction.
Fine-Tuning	Taking a pre-trained model that already understands general language and training it further on a specific dataset for a custom task.
Pipeline	A high-level, easy-to-use Hugging Face wrapper that handles tokenization, model execution, and post-processing in a single line of code.
Backbone	The primary core layers of a Transformer that extract patterns and meanings from raw text, without making a final task prediction.
Head	An extra layer added to the top of a backbone model designed for a specific task (e.g., a classification head or text generation head).
KV Caching	Storing past Key and Value matrices in RAM/VRAM during generation so the model doesn't recompute earlier tokens at every step.
Loss	A single number measuring how wrong the model's predictions are compared to the true labels (lower loss means better performance).

References

- Mainly LLM HF Course (Adapted).
- [Repository of this course](#)

Further Readings:

Pytorch :

1- <https://sebastianraschka.com/teaching/pytorch-1h/>

2- https://www.youtube.com/watch?v=I9_gp4tErHY

python :

[pythonRev_important_Libs](#)

[python cheatsheet](#)

Book : python for programmers ,Dietel